MANCHESTER 1824
The University of Manchester

COMP15111 Fundamentals of Computer Architecture

Addressing Modes

Pavlos Petoumenos
pavlos.petoumenos@manchester.ac.uk

Overview

- How RISC-V calculates addresses
- Multiple ways
- What is each way good for?

Memory (Recap)

- Ordered store of information
- Each location stores a value
- Each location is identified by a unique number
  - Address!

What's "addressing mode"?

- All memory instructions need to access a specific location → a specific address
- The process through which the processor generates the 32-bit address is the addressing mode
- Multiple mechanisms
  - Different kinds of data → different layout → different ways to calculate the address
  - No processor supports all of them (RISC-V supports fewer than usual!)
  - Parts of the addressing mechanism in software

Addressing Modes in RISC-V

- Absolute
- Register indirect
- Base-offset
- PC-relative

RISC-V Addressing

- Absolute
- Register indirect
- Base-offset
- PC-relative
- Register?
- Immediate?

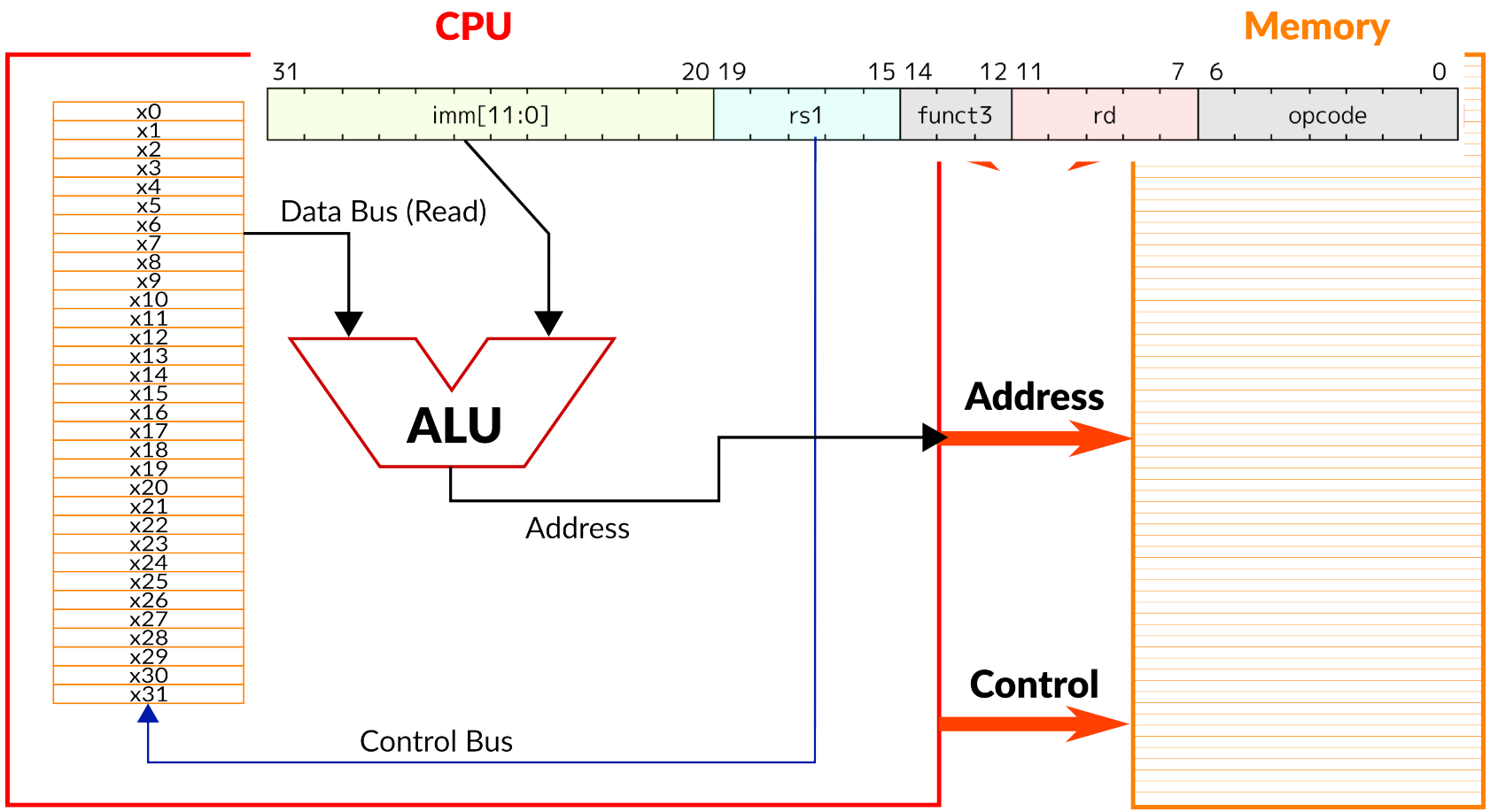
Pseudo-modes

Not really addressing modes

Base-offset Addressing

- The instruction encodes a base register and an offset relative to that register
  - lw rd, immediate[rs1]
  - sw rs2, immediate[rs1]
  - jalr rd, immediate[rs1]
  - addi rd, rs1, immediate (when part of address calculation)

Base-offset Addressing



Base-offset Addressing

- 12-bit immediate
  - -2,048 to +2,047
  - -4,096 to +4,095 for jalr
- What is it good for?
  - Base address is a variable (a pointer!)
  - Offset of specific data item in the data structure

Base-offset Example

Python

```
arr1 = [‘a’, ‘b’, ‘c’, ‘d’, …]

...

def find_third(arr):
  return arr[2]
```

Assembly

```
arr1 defb ‘a’, ‘b’, ‘c’, ‘d’,...

...

; find_third
; arr/return value in x10
  lb x10, 2[x10]
```

Another Example

Python

```
class Item:
  def __init__(self, val, next):
    self.name = “”
    self.val = val
    self.next = next

def accumulate_list(first_item):
  sum = 0
  item = first_item
  while item is not None:
    sum += item.val
    item = item.next
  return sum
```

Assembly

```
;Item layout
; name = offset 0
; val = offset 4
; next = offset 8

; accumulate_list
; first/return value in x10
  mv x11, x0
  beqz x10, exit
loop
  lw x5, 4[x10]
  add x11, x11, x5
  lw x10, 8[x10]
  bnez x10, loop
exit
  mv x10, x11
```

PC-relative Addressing

- Somewhat similar to Base-offset
  - But base address is the PC
  - Branches
  - jal
  - auipc

PC-relative Addressing

- What is it good for?
  - Jumps + branches tend to be local → small offsets relative to PC → offsets can be encoded in few bits
  - Relocatable code → As long as relative offsets remain constant, the code can be moved

PC-relative Addressing

```
loop
    auipc x10, variable ; offset of variable = 28
    sw x1, 28[x10]      ; x10 = pc of previous instr
    addi x1, x1, -1
    bgez x1, -12        ; loop
    li x17, 5
    ecall
variable defw 0
```

always +28

always -12

Absolute Addressing

- The address is directly given by the instruction
  - Impossible in the RISC-V ISA
  - Instructions are 32-bits & addresses are 32-bits
    - Possible in x86!
- Pseudo-instructions
  - lw, sw, and la
  - All of them translate into pc-relative addressing
- What is it good for?
  - Accessing one specific thing

Register Indirect Addressing

- The full address is in a register
  - Base-offset addressing with zero offset
- What is it good for?
  - All addressing algorithm in software

Other ISAs

- ARM → Base-offset, PC-relative, Register Indirect, Register Offset
  - Variants with increments and pre-scaling!
- X86-64 → 7 different modes*
- RISC-V
  - Only Base-offset and PC-relative
  - Absolute and Register Indirect → Specialisations of base-offset
  - Large offsets → Uncommon, implemented with pseudo-instructions
  - Increments, Scaling → Can be implemented in SW, with HW optimisations

Summary

- RISC-V ISA natively supports 2 addressing modes
- Base-offset: Register contains base address, immediate argument contains 12-bit offset
  - Access specific items in a struct or an array
- PC-relative: Base-offset but register is PC and is implied
  - Access individual items in a relocation-safe way
- Two other supported in the assembler
  - Absolute: Immediate only
  - Register Indirect: Register only
- Other modes implemented in software

MANCHESTER 1824
The University of Manchester

COMP15111 Fundamentals of Computer Architecture

Strings and Addressing Modes

Pavlos Petoumenos
pavlos.petoumenos@manchester.ac.uk

Overview

- Simple C-like strings
- Accessing strings
- Using addressing modes efficiently

Strings (Recap)

- Python strings
  msg = ‘Hello’

size=5	hash	‘H’	‘e’	‘l’	‘l’	‘o’

- C strings
  char* msg = “Hello”;

‘H’	‘e’	‘l’	‘l’	‘o’	0

- Assembly strings
  msg defb “Hello”, 0

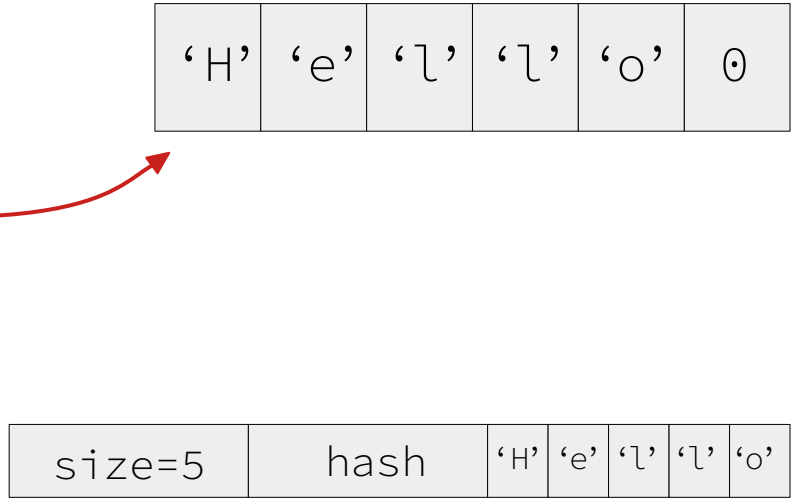
‘H’	‘e’	‘l’	‘l’	‘o’	0

Objects are (used through) their start address

- msg defb “Hello”, 0
  msg == symbol for the address of ‘H’

- char* msg = “Hello”;
  msg == address of ‘H’

- msg = ‘Hello’ →
  PyObject* msg →
  msg == address of size



String methods

- How do we operate on strings?
  - And other linear containers (vector, array, etc)
- len / find()

String length

- Pseudocode

```
i = 0
while(msg[i] is not 0):
  i += 1
length = i
```

- RISC-V assembly

```
    la x1, msg          ; x1 <-> msg
    mv x2, x0           ; x2 <-> i
loop
    add x3, x1, x2      ; msg[i] address
    lb x4, 0[x3]        ; msg[i] character
    addi x2, x2, 1      ; i = i + 1
    bnez x4, loop       ;

    subi x2, x2, 1      ; Why? Can we avoid this?
    sw x2, length, x1
```

String length

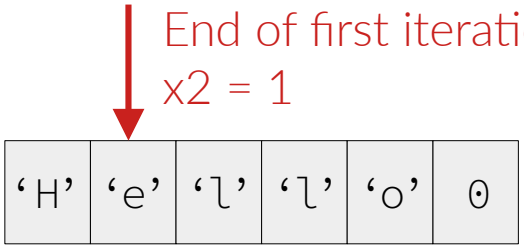
- Pseudocode

```
i = 0
while(msg[i] is not 0):
  i += 1
length = i
```

- RISC-V assembly

```
    la x1, msg          ; x1 <-> msg
    mv x2, x0           ; x2 <-> i
loop
    add x3, x1, x2      ; msg[i] address
    lb x4, 0[x3]        ; msg[i] character
    addi x2, x2, 1      ; i = i + 1
    bnez x4, loop       ;

    subi x2, x2, 1      ; Why? Can we avoid this?
    sw x2, length, x1
```



Length = 5

String length

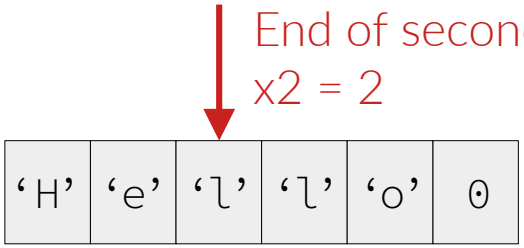
- Pseudocode

```
i = 0
while(msg[i] is not 0):
  i += 1
length = i
```

- RISC-V assembly

```
    la x1, msg          ; x1 <-> msg
    mv x2, x0           ; x2 <-> i
loop
    add x3, x1, x2      ; msg[i] address
    lb x4, 0[x3]        ; msg[i] character
    addi x2, x2, 1      ; i = i + 1
    bnez x4, loop       ;

    subi x2, x2, 1      ; Why? Can we avoid this?
    sw x2, length, x1
```



String length

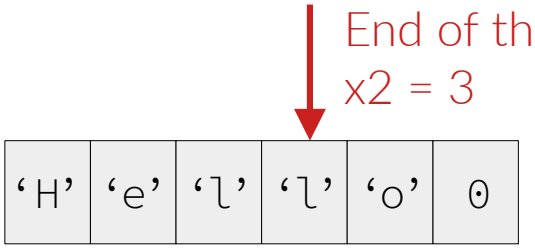
- Pseudocode

```
i = 0
while(msg[i] is not 0):
  i += 1
length = i
```

- RISC-V assembly

```
    la x1, msg          ; x1 <-> msg
    mv x2, x0           ; x2 <-> i
loop
    add x3, x1, x2      ; msg[i] address
    lb x4, 0[x3]        ; msg[i] character
    addi x2, x2, 1      ; i = i + 1
    bnez x4, loop       ;

    subi x2, x2, 1      ; Why? Can we avoid this?
    sw x2, length, x1
```



Length = 5

String length

- Pseudocode

```
i = 0
while(msg[i] is not 0):
  i += 1
length = i
```

- RISC-V assembly

```
    la x1, msg          ; x1 <-> msg
    mv x2, x0           ; x2 <-> i
loop
    add x3, x1, x2      ; msg[i] address
    lb x4, 0[x3]        ; msg[i] character
    addi x2, x2, 1      ; i = i + 1
    bnez x4, loop       ;

    subi x2, x2, 1      ; Why? Can we avoid this?
    sw x2, length, x1
```

End of fourth iteration
x2 = 4

'H'	'e'	'l'	'l'	'o'	0

Length = 5

String length

- Pseudocode

```
i = 0
while(msg[i] is not 0):
  i += 1
length = i
```

- RISC-V assembly

```
    la x1, msg          ; x1 <-> msg
    mv x2, x0           ; x2 <-> i
loop
    add x3, x1, x2      ; msg[i] address
    lb x4, 0[x3]        ; msg[i] character
    addi x2, x2, 1      ; i = i + 1
    bnez x4, loop       ;

    subi x2, x2, 1      ; Why? Can we avoid this?
    sw x2, length, x1
```

End of fifth iteration
x2 = 5

'H'	'e'	'l'	'l'	'o'	0

Length = 5

String length

- Pseudocode

```
i = 0
while(msg[i] is not 0):
  i += 1
length = i
```

- RISC-V assembly

```
    la x1, msg          ; x1 <-> msg
    mv x2, x0           ; x2 <-> i
loop
    add x3, x1, x2      ; msg[i] address
    lb x4, 0[x3]        ; msg[i] character
    addi x2, x2, 1      ; i = i + 1
    bnez x4, loop       ;

    subi x2, x2, 1      ; Why? Can we avoid this?
    sw x2, length, x1
```

End of sixth iteration
x2 = 6

'H'	'e'	'l'	'l'	'o'	0

Length = 5

String length

- Pseudocode

```
i = 0
while(msg[i] is not 0):
  i += 1
length = i
```

- RISC-V assembly

```
    la x1, msg          ; x1 <-> msg
    mv x2, x0           ; x2 <-> i
loop
    add x3, x1, x2      ; msg[i] address
    lb x4, 0[x3]        ; msg[i] character
    addi x2, x2, 1      ; i = i + 1
    bnez x4, loop       ;

    subi x2, x2, 1      ; Why? Can we avoid this?
    sw x2, length, x1
```

End of sixth iteration
x2 = 6

'H'	'e'	'l'	'l'	'o'	0

Length = **x2 – 1** = 5

String length

- Pseudocode

```
i = 0
while(msg[i] is not 0):
  i += 1
length = i
```

- RISC-V assembly

```
    la x1, msg          ; x1 <-> msg
    mv x2, x0           ; x2 <-> i
loop
    add x3, x1, x2      ; msg[i] address
    lb x4, 0[x3]        ; msg[i] character
    addi x2, x2, 1      ; i = i + 1
    bnez x4, loop       ;

    subi x2, x2, 1      ; Why? Can we avoid this?
    sw x2, length, x1
```

Register Indirect Addressing

Addressing logic implemented in software

String length

```
i = 0
while(msg[i] is not 0):
  i += 1
length = i
```

```
    la x1, msg          ; x1 <-> msg
    mv x2, x0           ; x2 <-> i
loop
    add x3, x1, x2      ; msg[i] address
    lb x4, 0[x3]        ; msg[i] character
    addi x2, x2, 1      ; i = i + 1
    bnez x4, loop       ;

    subi x2, x2, 1      ; Why? Can we avoid this?
    sw x2, length, x1
```

Register-Offset / Base+Index Addressing

ARM

```
ldrb r4, [r1,r2]
```

String length

```
i = 0
while(msg[i] is not 0):
  i += 1
length = i
```

```
    la x1, msg          ; x1 <-> msg
    mv x2, x0           ; x2 <-> i
loop
    add x3, x1, x2
    lb x4, 0[x3]
    addi x2, x2, 1
    bnez x4, loop

    subi x2, x2, 1
    sw x2, length, x1
```

2 fewer registers
One add less in the loop
1 machine instruction more

```
    la x1, msg          ; x1 <-> msg[i]

loop

    lb x2, 0[x1]
    addi x1, x1, 1
    bnez x2, loop
    la x2, msg
    sub x1, x1, x2
    subi x1, x1, 1
    sw x1, length, x2
```

x1 = address of msg[i]
x1 = msg + i
i = x1 - msg

String length

```
    la x1, msg          ; x1 <-> msg
    mv x2, x0           ; x2 <-> i
loop
    add x3, x1, x2
    lb x4, 0[x3]
    addi x2, x2, 1
    bnez x4, loop

    subi x2, x2, 1
    sw x2, length, x1
```

1 fewer register
One add less in the loop
Same machine instructions

```
i = 0
while(msg[i] is not 0):
  i += 1
length = i
```

```
    la x1, msg          ; x1 <-> msg[i]
    mv x3, x1
loop

    lb x2, 0[x1]
    addi x1, x1, 1
    bnez x2, loop

    sub x1, x1, x3
    subi x1, x1, 1
    sw x1, length, x2
```

String length

```
i = 0
while(msg[i] is not 0):
  i += 1
length = i
```

```
    la x1, msg        ; x1 <-> msg[i]
    mv x3, x1
loop

    lb x2, 0[x1]
    addi x1, x1, 1
    bnez x2, loop

    sub x1, x1, x3
    subi x1, x1, 1
    sw x1, length, x2
```

Post-incremented Register Indirect Adressing

ARM

```
ldrb r2, [r1], #1
```

String find()

- Index of the specified character
- msg.find('e')

```
        la x1, msg        ; x1 <-> msg[i]
        mv x3, x1
        li x4, 'e'
loop

        lb x2, 0[x1]
        addi x1, x1, 1
        bne x2, x4, loop

        sub x1, x1, x3
        subi x1, x1, 1
        sw x1, index, x2
```

What's wrong here?

String find()

- Index of the specified character
- msg.find('e')

What's wrong here?

```
        la x1, msg        ; x1 <-> msg[i]
        mv x3, x1
        li x4, 'e'
loop
        lb x2, 0[x1]
        addi x1, x1, 1
        bne x2, x4, loop

        sub x1, x1, x3
        subi x1, x1, 1
        sw x1, index, x2
```

Before the loop
x1 = msg + 0

'H'	'a'	'l'	'l'	'o'	0			

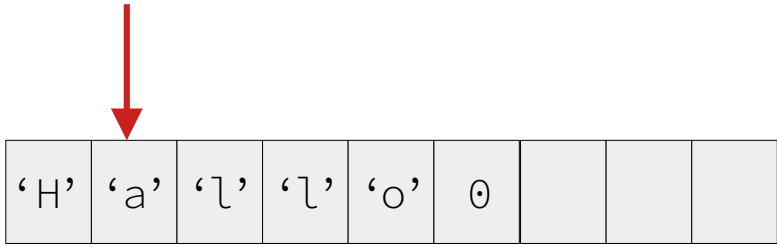
String find()

- Index of the specified character
- msg.find('e')

What's wrong here?

```
        la x1, msg        ; x1 <-> msg[i]
        mv x3, x1
        li x4, 'e'
loop
        lb x2, 0[x1]
        addi x1, x1, 1
        bne x2, x4, loop

        sub x1, x1, x3
        subi x1, x1, 1
        sw x1, index, x2
```



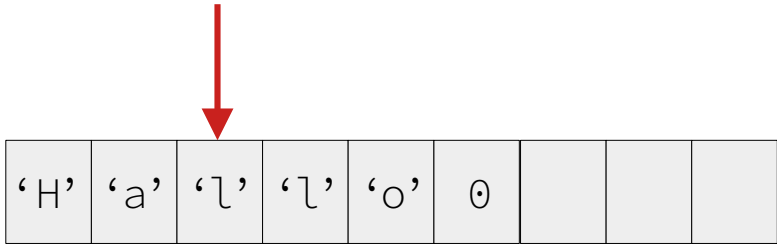
String find()

- Index of the specified character
- msg.find('e')

What's wrong here?

```
        la x1, msg        ; x1 <-> msg[i]
        mv x3, x1
        li x4, 'e'
loop
        lb x2, 0[x1]
        addi x1, x1, 1
        bne x2, x4, loop

        sub x1, x1, x3
        subi x1, x1, 1
        sw x1, index, x2
```



String find()

- If the character is not in the string
  - Program needs to terminate
  - Python find() returns -1

```
    la x1, msg        ; x1 <-> msg[i]
    mv x3, x1
    li x4, ‘e’

loop
    lb x2, 0[x1]
    addi x1, x1, 1
    bne x2, x4, loop

    sub x1, x1, x3
    subi x1, x1, 1
    sw x1, index, x2
```

String find()

- If the character is not in the string
  - Program needs to terminate
  - Python find() returns -1

```
    la x1, msg          ; x1 <-> msg[i]
    mv x3, x1
    li x4, 'e'

loop
    lb x2, 0[x1]
    addi x1, x1, 1
    beq x2, x4, exit
    bnez x2, loop

exit

    sub x1, x1, x3
    subi x1, x1, 1
    sw x1, index, x2
```

String find()

- What if the character is not in the string
  - Python find() returns -1

```
    la x1, msg          ; x1 <-> msg[i]
    mv x3, x1
    li x4, 'e'

loop
    lb x2, 0[x1]
    addi x1, x1, 1
    beq x2, x4, exit
    bnez x2, loop

exit
    bez x2, notfound
    sub x1, x1, x3
    subi x1, x1, 1
    sw x1, length, x2
    j end
notfound
    li x1, -1
    sw x1, index, x2
end
```

Here only if not found

```
    la x1, msg          ; x1 <-> msg[i]
    mv x3, x1
    li x4, 'e'

loop
    lb x2, 0[x1]
    addi x1, x1, 1
    beq x2, x4, exit
    bnez x2, loop

exit
    bez x2, notfound
    sub x1, x1, x3
    subi x1, x1, 1
    sw x1, length, x2
    j end
notfound
    li x1, -1
    sw x1, index, x2
end
```

-1 instruction
Found path → -2 instructions
Not found path → Same instructions

```
    la x1, msg          ; x1 <-> msg[i]
    mv x3, x1
    li x4, 'e'

loop
    lb x2, 0[x1]
    addi x1, x1, 1
    beq x2, x4, exit
    bnez x2, loop

    li x1, -1
    sw x1, length, x2
    j end

exit
    sub x1, x1, x3
    subi x1, x1, 1
    sw x1, index, x2
end
```

```
    la x1, msg
    mv x3, x1
    li x4, 'e'

loop
    lb x2, 0[x1]
    addi x1, x1, 1
    beq x2, x4, exit
    bnez x2, loop

    li x1, -1
    sw x1, length, x2
    j end

exit
    sub x1, x1, x3
    subi x1, x1, 1
    sw x1, index, x2
end
```

index = x1 – msg – 1
for index = -1 → x1
= msg

-2 instructions
Found path → Same
Not found path →
One jump less, two
subs more

```
    la x1, msg          ; x1 <-> msg[i]
    mv x3, x1
    li x4, 'e'

    lb x2, 0[x1]
    addi x1, x1, 1
    beq x2, x4, exit
    bnez x2, loop

    mv x1, x3

exit
    sub x1, x1, x3
    subi x1, x1, 1
    sw x1, index, x2
```

Summary

- Operating on strings (or other linear data)
  - Base + Index
  - Post-increment Register Indirect
  - Software implemented on RISC-V
  - Natively supported on ARM

MANCHESTER 1824
The University of Manchester

COMP15111 Fundamentals of Computer Architecture

Tables

Pavlos Petoumenos
pavlos.petoumenos@manchester.ac.uk

Overview

- What is a table?
- How to use it efficiently?

Colours

- Bitmap Images
  - Each pixel coloured independently
  - Devices, image files (compressed or uncompressed)
  - Each pixel can have a large number of colours
- Black and White → Two colours → 1 bit per pixel
- Greyscale → 256 shades of gray → 8 bits
- CGA (1981) → 16 colours → 4 bits
- VGA (1987) → 256 colours → 8 bits
- SVGA+ (including Web) → 16M colours → 24 bits: Red, Green, Blue intensities each 8-bits (0-255)

Palettes

- Using a subset of the possible colours
  - Fewer colours → fewer bits
  - Fewer bits → Lower storage + Fewer physical wires
- VGA Mode 13h: 256 colours out of 2^{18} (256k)
- xterm: 16 colours (8 colours with bright and dark variations)
- Indexed PNG: 2-256 colours (1-8 bits)

Display	Xterm Number	Xterm Name	HEX	RGB	HSL
	0	Black (SYSTEM)	#000000	rgb(0,0,0)	hsl(0,0%,0%)
	1	Maroon (SYSTEM)	#800000	rgb(128,0,0)	hsl(0,100%,25%)
	2	Green (SYSTEM)	#008000	rgb(0,128,0)	hsl(120,100%,25%)
	3	Olive (SYSTEM)	#808000	rgb(128,128,0)	hsl(60,100%,25%)
	4	Navy (SYSTEM)	#000080	rgb(0,0,128)	hsl(240,100%,25%)
	5	Purple (SYSTEM)	#800080	rgb(128,0,128)	hsl(300,100%,25%)
	6	Teal (SYSTEM)	#008080	rgb(0,128,128)	hsl(180,100%,25%)
	7	Silver (SYSTEM)	#c0c0c0	rgb(192,192,192)	hsl(0,0%,75%)
	8	Grey (SYSTEM)	#808080	rgb(128,128,128)	hsl(0,0%,50%)
	9	Red (SYSTEM)	#ff0000	rgb(255,0,0)	hsl(0,100%,50%)
	10	Lime (SYSTEM)	#00ff00	rgb(0,255,0)	hsl(120,100%,50%)
	11	Yellow (SYSTEM)	#ffff00	rgb(255,255,0)	hsl(60,100%,50%)
	12	Blue (SYSTEM)	#0000ff	rgb(0,0,255)	hsl(240,100%,50%)
	13	Fuchsia (SYSTEM)	#ff00ff	rgb(255,0,255)	hsl(300,100%,50%)
	14	Aqua (SYSTEM)	#00ffff	rgb(0,255,255)	hsl(180,100%,50%)
	15	White (SYSTEM)	#ffffff	rgb(255,255,255)	hsl(0,0%,100%)

256 Colors Cheat Sheet - Xterm system colors, HEX, RGB, HSL

© 256 colors cheat sheet by Jonas Jared Jacek is licensed under CC BY-NC-SA 4.0.

xterm palette

Number	RGB in hex	Name
0	#00 00 00	black
1	#80 00 00	maroon
2	#00 80 00	green
3	#80 80 00	olive
4	#00 00 80	navy
5	#80 00 80	purple
6	#00 80 80	teal
7	#c0 c0 c0	silver
8	#80 80 80	grey

- 16 colours → 4 bits to identify the colour
- Each 4-bit id corresponds to the 24-bit RGB colour
  -

xterm palette: ls output

```
ppetou@sfaktiria4:~/live/arch/week2$ ls -la
total 42976
drwxrwxr-x 3 ppetou ppetou     4096 Oct  3 12:35 .
drwxrwxr-x 4 ppetou ppetou     4096 Oct  2 08:53 ..
-rw-rw-r-- 1 ppetou ppetou      562 Oct  2 13:52 discount.s
-rw-rw-r-- 1 ppetou ppetou      513 Oct  2 13:38 discount.s~
-rw-rw-r-- 1 ppetou ppetou     1947 Oct  2 13:53 discount.s.kmd
-rwxrw-r-- 1 ppetou ppetou        0 Oct  3 12:34 script.sh
-rw-rw-r-- 1 ppetou ppetou 43974688 Oct  3 12:34 specs.tar.xz
drwxrwxr-x 2 ppetou ppetou     4096 Oct  3 12:35 test
ppetou@sfaktiria4:~/live/arch/week2$
```

Colour 12 (blue)

Colour 9 (red)

Colour 2 (lime)

Colour 0 (black)

xterm palette: ls output

Colour 12

Colour 9

Colour 2

Colour 0

```
total 42976
drwxrwxr-x 3 ppetou ppetou     4096 Oct  3 12:35 .
drwxrwxr-x 4 ppetou ppetou     4096 Oct  2 08:53 ..
-rw-rw-r-- 1 ppetou ppetou      562 Oct  2 13:52 discount.s
-rw-rw-r-- 1 ppetou ppetou      513 Oct  2 13:38 discount.s~
-rw-rw-r-- 1 ppetou ppetou     1947 Oct  2 13:53 discount.s.kmd
-rwxrw-r-- 1 ppetou ppetou        0 Oct  3 12:34 script.sh
-rw-rw-r-- 1 ppetou ppetou 43974688 Oct  3 12:34 specs.tar.xz
drwxrwxr-x 2 ppetou ppetou     4096 Oct  3 12:35 test
ppetou@sfaktiria4:~/live/arch/week2$
```

xterm palette

Number	RGB in hex	Name
0	#00 00 00	black
1	#80 00 00	maroon
2	#00 80 00	green
3	#80 80 00	olive
4	#00 00 80	navy
5	#80 00 80	purple
6	#00 80 80	teal
7	#c0 c0 c0	silver
8	#80 80 80	grey

- A table of 16 colours mapping to 24-bit colour values
- 16 colours → 4 bits to identify the colour
- 4 bits → line in the table
  - $0101_2 = 5_{10}$ means #800080

Colour lookup

```
ppetou@sfaktiria4:~/live/arch/week2$ ls -la
total 42976
drwxrwxr-x 3 ppetou ppetou     4096 Oct  3 12:35 .
drwxrwxr-x 4 ppetou ppetou     4096 Oct  2 08:53 ..
-rw-rw-r-- 1 ppetou ppetou      562 Oct  2 13:52 discount.s
-rw-rw-r-- 1 ppetou ppetou      513 Oct  2 13:38 discount.s~
-rw-rw-r-- 1 ppetou ppetou     1947 Oct  2 13:53 discount.s.kmd
-rwxrw-r-- 1 ppetou ppetou        0 Oct  3 12:34 script.sh
-rw-rw-r-- 1 ppetou ppetou 43974688 Oct  3 12:34 specs.tar.xz
drwxrwxr-x 2 ppetou ppetou     4096 Oct  3 12:35 test
ppetou@sfaktiria4:~/live/arch/week2$
```

- Colour info
  → Palette id (4-bits)
- OS and display
  → RGB (24-bits)
- Common operation
  - Paint a display pixel based on the colour of an image pixel or an xterm pixel
  - Which 24-bit colour corresponds to 4-bit representation?

Colour lookup

- 4 instructions for each colour
- Executes on average
  - 16/2 branches
  - 16/2 adds
  - + 3 other instructions

```
; 4-bit colour ID in x10
    mv x1, x0
    bne x10, x1, skip0  ; if x10 == 0
    li x10, 0x000000
    j exit
skip0
    addi x1, x1, 1
    bne x10, x1, skip1  ; elif x10 == 1
    li x10, 0x800000
    j exit
skip1
    addi x1, x1, 1
    bne x10, x2, skip2  ; elif x10 == 2
    li x10, 0x008000
    j exit
skip2
    ...
```

Table Lookup

Number	RGB in hex	Name
0	#00 00 00	black
Row = 2	#80 00 00	maroon
2	#00 80 00	green
3	#80 80 00	olive
4	#00 00 80	navy
5	#80 00 80	purple
6	#00 80 80	teal
7	#c0 c0 c0	silver
8	#80 80 80	grey

offset = 8

```
table  defw 0x000000 ; black
       defw 0x800000 ; maroon
       defw 0x008000 ; green
       defw 0x808000 ; olive
       defw 0x000080 ; navy
       defw 0x800080 ; purple
       defw 0x008080 ; teal
       defw 0xc0c0c0 ; silver
```

Table lookup

- Memory
  - 6 machine instructions in total
  - 1 word per entry
  - Less storage for more than 2 entries
- Executes 6 instructions
  - Faster for more than 2-3 entries

```
; 4-bit colour ID in x10
    la x1, table      ; table address in x1
    li x2, 4          ; 1 word = 4 bytes
    mul x2, x10, x2   ; ID * 4 = byte offset
    add x1, x1, x2    ; table + offset
    lw x10, 0[x1]     ; load colour
    ...

table  defw 0x000000 ; black
       defw 0x800000 ; maroon
       defw 0x008000 ; green
       defw 0x808000 ; olive
       defw 0x000080 ; navy
       defw 0x800080 ; purple
       defw 0x008080 ; teal
       defw 0xc0c0c0 ; silver
```

Table lookup

- M
  - Base + index + prescaling addressing
  - instruc
  - Faster for more than 2-3 entries

```
; 3-bit colour ID in x10
  la x1, table      ; table address in x1
  li x2, 4          ; 1 word = 4 bytes
  mul x2, x10, x2   ; ID * 4 = byte offset
  add x1, x1, x2    ; table + offset
  lw x10, 0[x1]     ; load colour
  ...

le  defw 0x000000 ; black
    defw 0x800000 ; maroon
    defw 0x008000 ; green
    defw 0x808000 ; olive
    defw 0x000080 ; navy
    defw 0x800080 ; purple
    defw 0x008080 ; teal
    defw 0xc0c0c0 ; silver
```

Table lookup

- M
  ries
- Executes 6 instructions
  - Faster for more than 2-3 entries

Multiplication is expensive!

```
; 3-bit colour ID in x10
  la x1, table      ; table address in x1
  li x2, 4          ; 1 word = 4 bytes
  mul x2, x10, x2   ; ID * 4 = byte offset
  add x1, x1, x2    ; table + offset
  lw x10, 0[x1]     ; load colour
  ...

table  defw 0x000000 ; black
       defw 0x800000 ; maroon
       defw 0x008000 ; green
       defw 0x808000 ; olive
       defw 0x000080 ; navy
       defw 0x800080 ; purple
       defw 0x008080 ; teal
       defw 0xc0c0c0 ; silver
```

Summary

- Tables: list of values in memory
- id → offset from start → value
- Table lookup
  - Efficiently translate one representation to another
  - Base + index * value size

MANCHESTER 1824
The University of Manchester

COMP15111 Fundamentals of Computer Architecture

Shift Operations

Pavlos Petoumenos
pavlos.petoumenos@manchester.ac.uk

Overview

- Shift operations
- Move bits left/right
- Efficient implementation of complex operations

Data Processing Instructions (Recap)

	Arithmetic	Logic	Shift	Comparison
Register only	`add` → Addition	`and` → Bitwise AND	`sll` → Shift left logical	
	`sub` → Subtraction	`or` → Bitwise OR	`srl` → Shift right logical	
	`mul{..}`→Multiply No option → lower half of 64-bit result h → upper half (signed) hu → upper half (unsigned) hsu → upper half (signed * unsigned)	`xor` → Bitwise XOR	`sra` → Shift right arithmetic	`slt{u}` → set if rs1 < rs2
	`div{u}` → Division (optional u indicates unsigned)			
	`rem{u}` → Remainder (optional u indicates unsigned)			
Immediate versions	`addi`	`andi`	`slli`	`slti{u}`
	~~`subi`~~ (pseudo)	`ori`	`srli`	
		`xori`	`srai`	

Data Processing Instructions (Recap)

	Arithmetic	Logic	Shift	Comparison
Register only	`add` → Addition	`and` → Bitwise AND	`sll` → Shift left logical	
	`sub` → Subtraction	`or` → Bitwise OR	`srl` → Shift right logical	
	`mul{..}`→Multiply No option → lower half of 64-bit result h → upper half (signed) hu → upper half (unsigned) hsu → upper half (signed * unsigned)	`xor` → Bitwise XOR	`sra` → Shift right arithmetic	`slt{u}` → set if rs1 < rs2
	`div{u}` → Division (optional u indicates unsigned)			
	`rem{u}` → Remainder (optional u indicates unsigned)			
Immediate versions	`addi`	`andi`	`slli`	`slti{u}`
	~~`subi`~~ (pseudo)	`ori`	`srli`	
		`xori`	`srai`	

Shift Operations

- Move the bits inside the register
  - `sll`: Shift Left Logical
  - `srl`: Shift Right Logical
  - `sra`: Shift Right Arithmetic
- mnemonic rd, rs1, rs2
  - rs1: the value to modify
  - rs2: how many bits to move

Logical Left Shift

- Increase the significance of each bit

Input:	x_7	x_6	x_5	x_4	x_3	x_2	x_1	x_0
Left shifted by one place	x_6	x_5	x_4	x_3	x_2	x_1	x_0	0

e.g. $0000\ 1111_2$ becomes $0001\ 1110_2$
15_{10} becomes 30_{10}

- Effectively multiplying by two!
  - Shifting decimal digits left (e.g. 85 → 850), same as multiplying by ten
  - Shifting binary digits left, same as multiplying by 2
  - Shifting n bits left, same as multiplying by 2^n

Logical Right Shift

- Decrease the significance of each bit

Input:	x_7	x_6	x_5	x_4	x_3	x_2	x_1	x_0
Right shifted by one place	0	x_7	x_6	x_5	x_4	x_3	x_2	x_1

e.g. $0000\ 1111_2$ becomes $0000\ 0111_2$
15_{10} becomes 7_{10}

- Effectively dividing by two!
  - Shifting decimal digits right (e.g. 120 → 12), same as dividing by ten
  - Shifting binary digits right, same as dividing by 2
  - Shifting n bits right, same as dividing by 2^n

Shifting numbers: Left

- $0000\ 1000_2 \rightarrow 0001\ 0000_2$
  - Unsigned/Signed: 8 → 16 ✓
- $1000\ 1000_2 \rightarrow 0001\ 0000_2$
  - Unsigned: 136 → 16 ✗
  - Signed: -120 → 16 ✗
- $1100\ 1000_2 \rightarrow 1001\ 0000_2$
  - Unsigned: 200 → 144 ✗
  - Signed: -56 → -112 ✓

Shift left is multiplication by 2 if the result fits in the register

Signedness is irrelevant

Shifting numbers: Right

- $0000\ 1000_2 \rightarrow 0000\ 0100_2$
  - Unsigned/Signed: 8 → 4 ✓
- $1000\ 1000_2 \rightarrow 0100\ 0100_2$
  - Unsigned: 136 → 68 ✓
  - Signed: -120 → 68 ✗
- $1100\ 1000_2 \rightarrow 0110\ 0100_2$
  - Unsigned: 200 → 100 ✓
  - Signed: -56 → 100 ✗

Shift right is division by 2 if the number is unsigned or positive

Signedness matters!

Shifting Right for signed numbers

- sra: Shift Right Arithmetic

Input:	x_7	x_6	x_5	x_4	x_3	x_2	x_1	x_0
Right shifted by one place	x_7	x_7	x_6	x_5	x_4	x_3	x_2	x_1

Shift right in a way that makes sense arithmetically assuming the number is signed, i.e. it divides by two

- $0100\ 0100_2 \rightarrow 0010\ 0010_2$
  - $68 \rightarrow 34$
- $1000\ 0100_2 \rightarrow 1100\ 0010_2$
  - $-124 \rightarrow -62$

Table lookup

- M
  ries
- Executes 6 instructions
  - Faster for more than 2-3 entries

Multiplication is expensive!

```
; 3-bit colour ID in x10
  la x1, table      ; table address in x1
  li x2, 4          ; 1 word = 4 bytes
  mul x2, x10, x2   ; ID * 4 = byte offset
  add x1, x1, x2    ; table + offset
  lw x10, 0[x1]     ; load colour
  ...

table  defw 0x000000 ; black
       defw 0x800000 ; maroon
       defw 0x008000 ; green
       defw 0x808000 ; olive
       defw 0x000080 ; navy
       defw 0x800080 ; purple
       defw 0x008080 ; teal
       defw 0xc0c0c0 ; silver
```

Table lookup

- M
  - ies
- Executes 6 instructions
  - Faster for more than 2-3 entries

Shifting is cheap!

```
; 3-bit colour ID in x10
  la x1, table      ; table address in x1
  li x2, 2          ; 1 word = 4 bytes → 2²
  sll x2, x10, x2   ; ID * 4 = byte offset
  add x1, x1, x2    ; table + offset
  lw x10, 0[x1]     ; load colour
  ...

table  defw 0x000000 ; black
       defw 0x800000 ; maroon
       defw 0x008000 ; green
       defw 0x808000 ; olive
       defw 0x000080 ; navy
       defw 0x800080 ; purple
       defw 0x008080 ; teal
       defw 0xc0c0c0 ; silver
```

Table lookup

- [illegible]
- Executes 6 instructions
  - Faster for more than 2-3 entries

Shifting works with immediates

```
; 3-bit colour ID in x10
  la x1, table      ; table address in x1

  slli x2, x10, 2   ; ID * 4 = byte offset
  add x1, x1, x2    ; table + offset
  lw x10, 0[x1]     ; load colour
  ...

table  defw 0x000000 ; black
       defw 0x800000 ; maroon
       defw 0x008000 ; green
       defw 0x808000 ; olive
       defw 0x000080 ; navy
       defw 0x800080 ; purple
       defw 0x008080 ; teal
       defw 0xc0c0c0 ; silver
```

Summary

- Shift left
  - Move bits left
  - Increase bit significance
  - Multiply by 2 (if result fits)
- Shift right
  - Move bits right
  - Decrease bit significance
  - Divide by 2 (if positive or using arithmetic shifting)